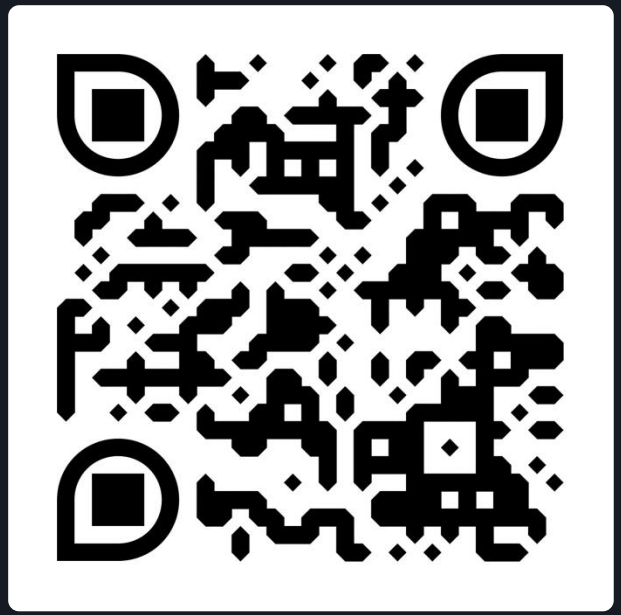
Stop Polling Your Database for AI Context. Use CDC.

>_ Serhat Kayikci — Senior Backend Engineer, Kafka & CDC Specialist

LLMday Hamburg, June 18, 2026

>_ Whoami?

- Senior Backend & Platform Engineer
- 15+ years — Kafka, CDC, event-driven architecture
- Currently: Mercedes-Benz AWS infrastructure @ iC Consult
- Freelance: Kafka & CDC consulting (Hossted)



```
>_ Somewhere right now, a chatbot is confidently recommending a product that was discontinued six months ago.
```

The model is fine. The vector database is fine. The data is stale.

>_ RAG in 30 seconds



The retrieval step is only as fresh as your last update.

>_ Your RAG pipeline has a time bomb



8h. 24h. 48h. Every answer comes from a snapshot of the past.

>_ Why not just poll faster?

- Every poll is a full query — and **99% of them return nothing new**
- Tighter loops don't buy freshness, they **hammer your production database**
- Best case: minutes of lag — and you pay the query cost **24/7**

Polling trades database load for freshness. CDC gives you both.

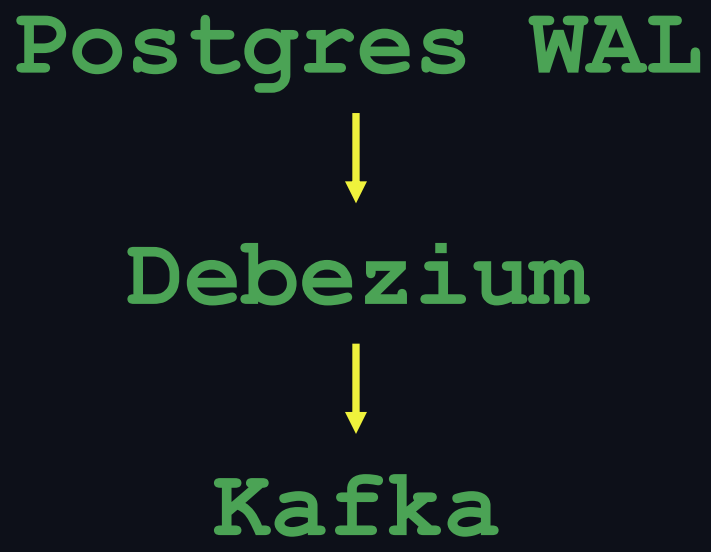
>_ Streaming engineers solved this years ago

CDC = Change Data Capture

Watch the Write-Ahead Log, not the table.

Every INSERT, UPDATE, DELETE becomes an event, instantly.

>_ How CDC works



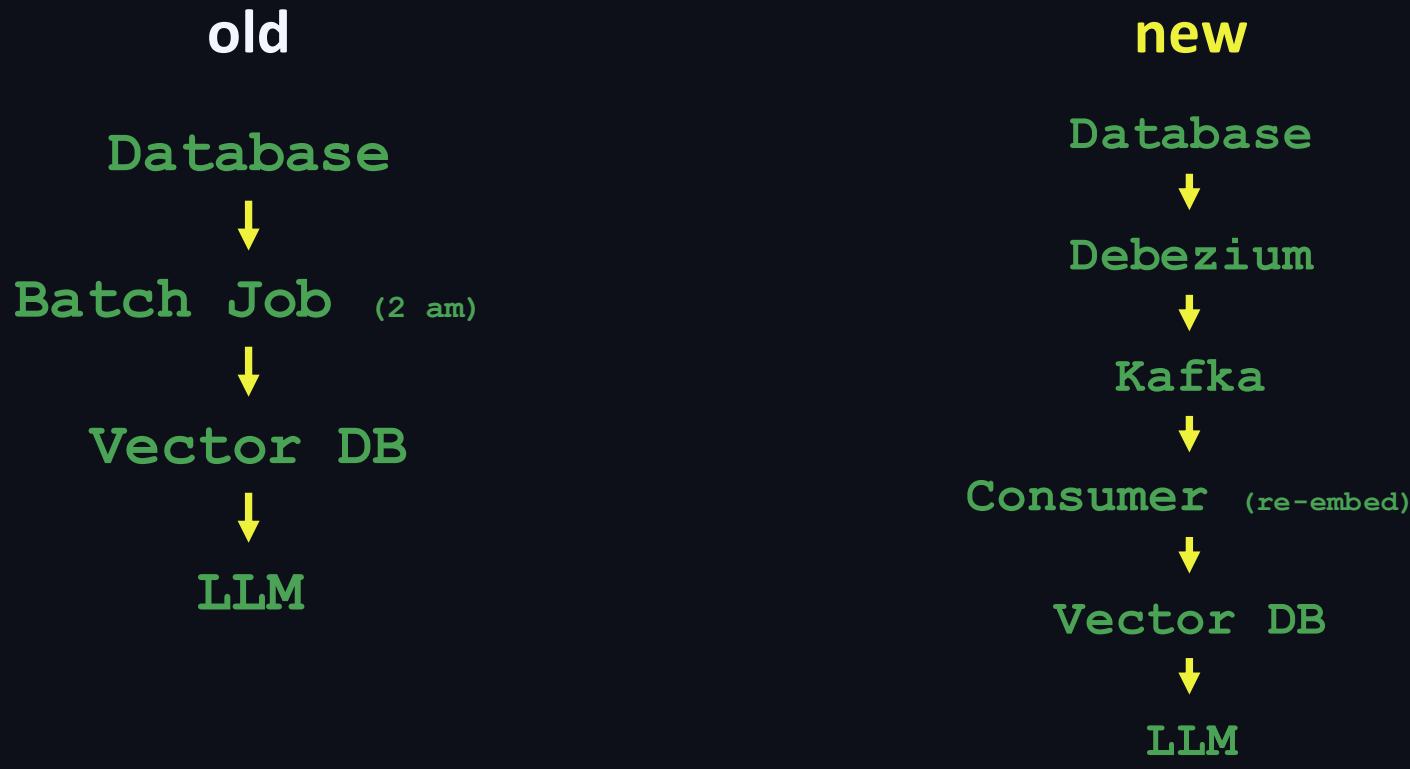
No polling. No batch. Just events.

>_ Anatomy of a change event

```
{
  "op": "u",    // u = update, c = create, d = delete
  "before": { "id": 4711, "price": 19.99, "status": "active" },
  "after":  { "id": 4711, "price": 14.99, "status": "discontinued" },
  "ts_ms": 1750241830123
}
```

Before & after in one payload — the consumer knows exactly what changed.

>_ The architecture



Only changed rows are re-embedded — the vector DB stays current.

>_ Batch vs CDC

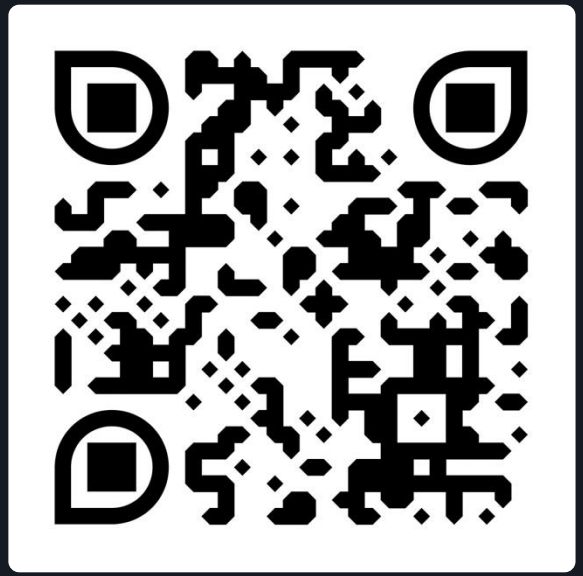
	BATCH	CDC
Freshness	hours behind	< 1 second
Database load	full table scans	reads the WAL
Failure mode	silently stale	replayable events
Re-index	everything, every night	only what changed

>_ What we're using today

PostgreSQL	Debezium	Kafka	Gemini
Relational Database	Change Data Capture	Event Streaming	Google's AI

Every layer is swappable — the pattern is vendor-neutral.

>_ DEMO



>_ What just happened

- SQL **UPDATE** → Kafka event in **< 1 second**
- Consumer updated context automatically
- LLM answered correctly — no restart, no re-index

>_ The happy path is easy. Production is not.

- Connector goes **FAILED** silently → monitor with heartbeat topics
- Schema evolution → a new column breaks your consumer
- Column names with spaces → Debezium regex limitation *(real story)*

>_ More production realities

- At-least-once delivery → your consumer must be **idempotent**
- Initial snapshot on large tables → use `snapshot.mode=schema_only`
- Managed Kafka (GCP, AWS MSK) → custom connector support varies

>_ When does this pattern fit?

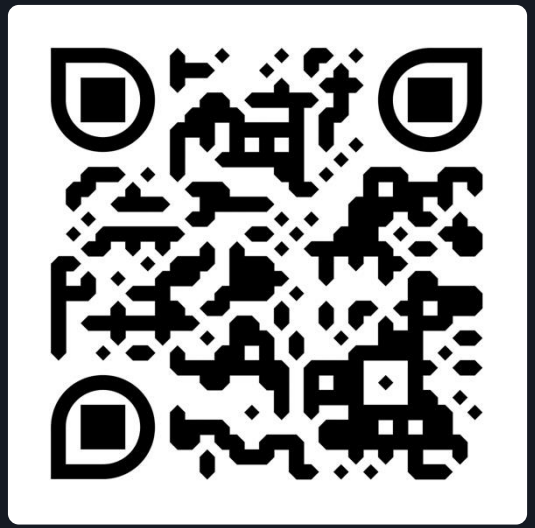
- **Good fit:** support chatbots, inventory-aware AI, real-time personalization
- **Overkill:** static reference data — batch is perfectly fine
- **Price of admission:** Kafka + Debezium ops, monitoring, schema management

>_ Your AI is only as smart as your freshest data.

Stream it. Don't poll it.

>_ Building AI on live data? Let's talk.

>_ LinkedIn



>_ Demo

